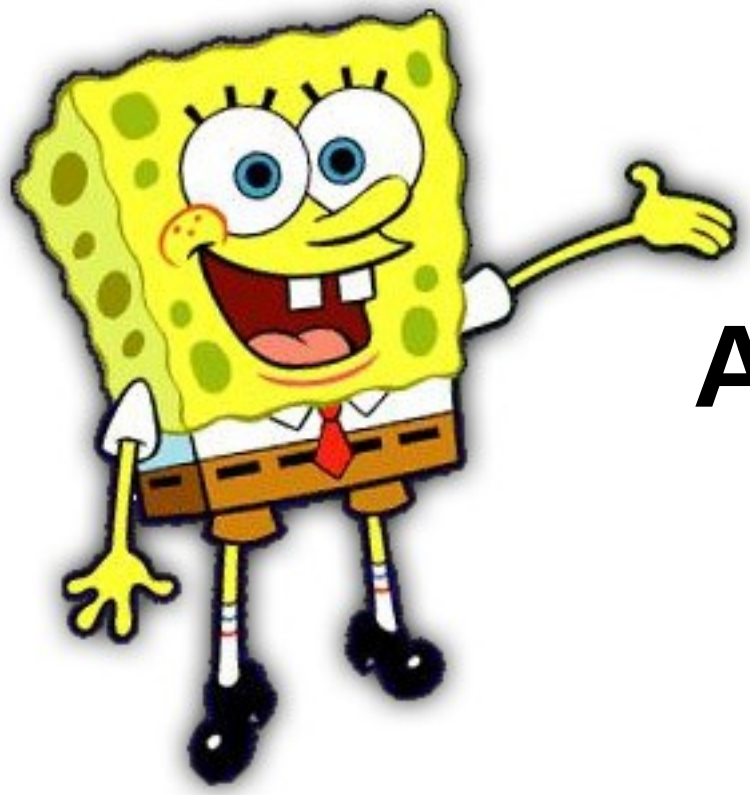


Viviana Mascardi, University of Genova, invited talk given at the International Conference on Logic Programming in 2012



Agents and MASs



Weak notion

An agent is an hardware or software system

- situated,
- autonomous,
- flexible
  - reactive
  - proactive
  - social.

N. Jennings, K. Sycara, M. Wooldridge, JAAMAS 1(1), 1998.

Strong notion



A computer system either conceptualized or implemented using concepts that are more usually applied to humans.

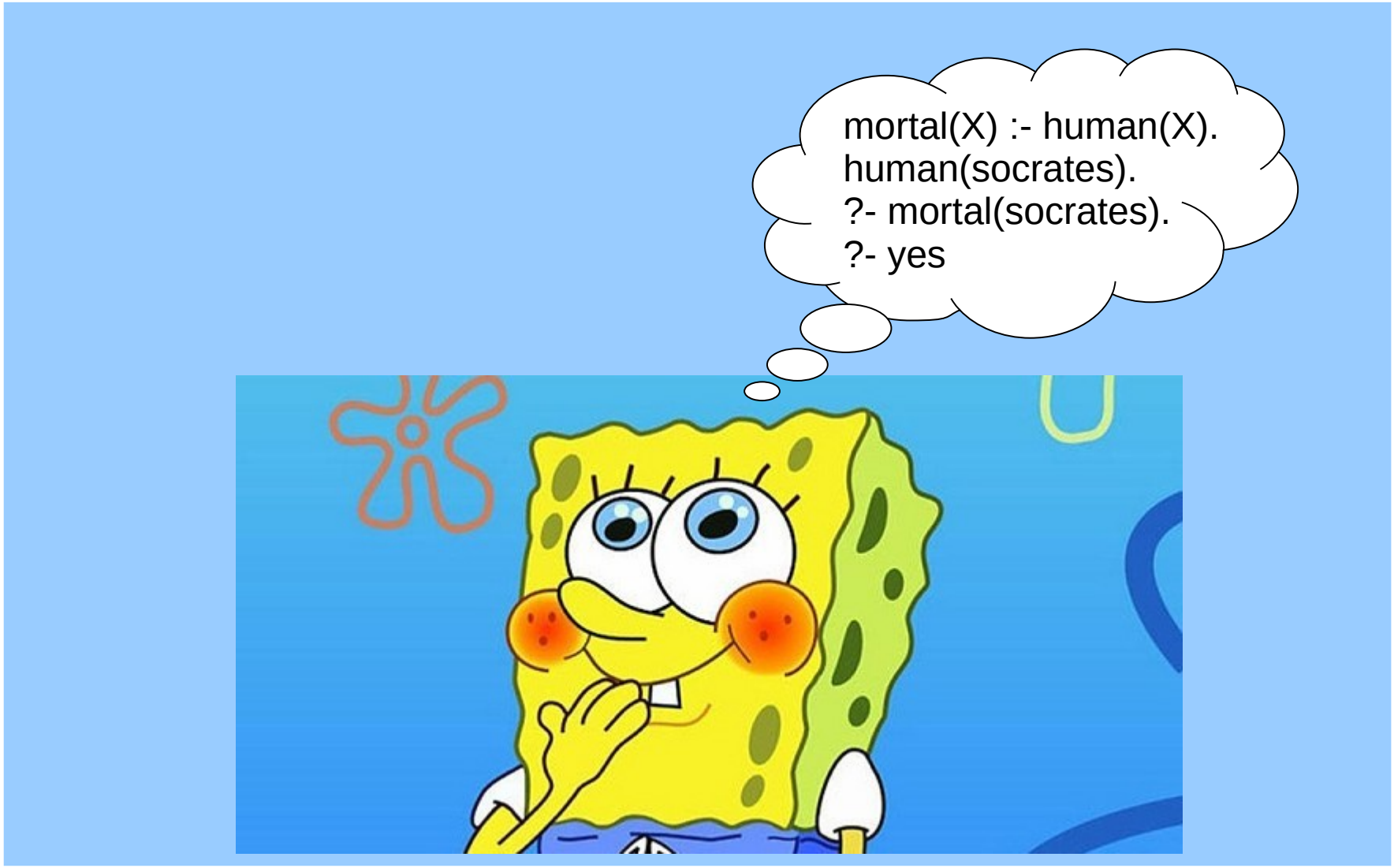
- Mentalistic notions.

Y. Shoham. Agent-oriented programming. Artificial Intelligence, 60(1):51–92, 1993.

- Emotional notions.

J. Bates. The role of emotion in believable agents. Communications of the ACM, 37(7):122–125, July 1994.

Logic-based agents



AgentSpeak(L)



The BDI architecture is one of the best known and most studied architectures for cognitive agents.

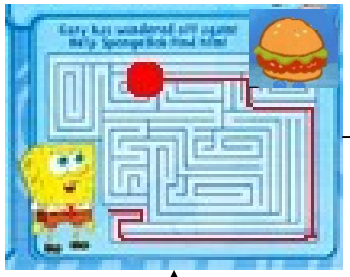
AgentSpeak(L) is an elegant, logic-based programming language inspired by the BDI architecture.

Anand S. Rao. AgentSpeak(L): BDI agents speak out in a logical computable language. In Proc. of MAAMAW '96, pp. 42-55, 1996.

The BDI architecture



Plans



Intentions



Reasoner



Goals



Beliefs



Environment

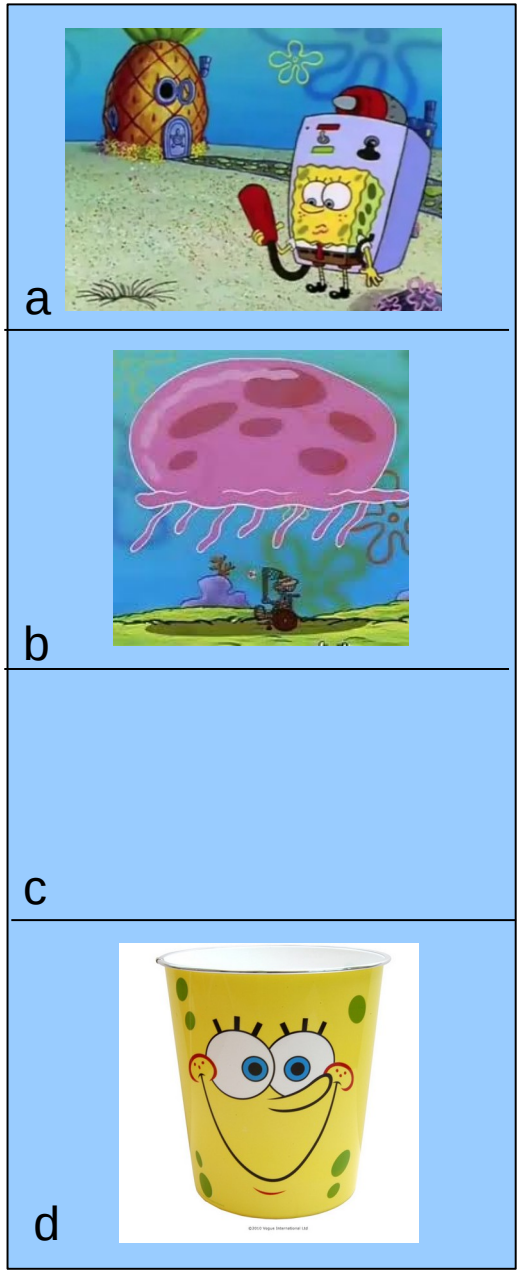
AgentSpeak(L) syntax

$$
\begin{array}{lll}
ag & ::= & bs \quad ps \\
bs & ::= & at_1 \ldots at_n \qquad (n \geq 0) \\
at & ::= & P(t_1, \ldots t_n) \qquad (n \geq 0) \\
ps & ::= & p_1 \ldots p_n \qquad (n \geq 1) \\
p & ::= & te : ct \leftarrow h \\
te & ::= & +at \mid -at \mid +g \mid -g \\
ct & ::= & at \mid \neg at \mid ct \ \wedge \ ct \mid \mathsf{T} \\
h & ::= & a \mid g \mid u \mid h; h \\
g & ::= & !at \mid \ ?at \\
u & ::= & +at \mid -at
\end{array}
$$

Jelly-fish busters

/* Initial beliefs */

adjacent(a, b).
adjacent(b, c).
adjacent(c, d).
location(myself, a).
location(jellyfish, b).
location(bin, d).



Jelly-fish busters

/* Plan 1 */

```
+!bust_jellyfishes :
   location(myself,X) &
   location(jellyfish,X) &
   location(bin,Y)
 <- !pick(jellyfish, X);
     move_to(Y);
     !drop(jellyfish, Y).
```

a



b



c

d



Jelly-fish busters

/* Plan 2 */

```
+!bust_jellyfishes :
   location(myself,X) &
   not location(jellyfish,X) &
   adjacent(X, Z)
 <- move_to(Z);
     !bust_jellyfishes.
```

a



b



c

d



Jelly-fish busters

/* Initial goal */

!bust_jellyfishes.

a



b



c

d



Jelly-fish busters

/* Initial beliefs */

```
adjacent(a, b).
adjacent(b, c).
adjacent(c, d).
location(myself, a).
location(jellyfish, b).
location(bin, d).
```

/* Initial goal */

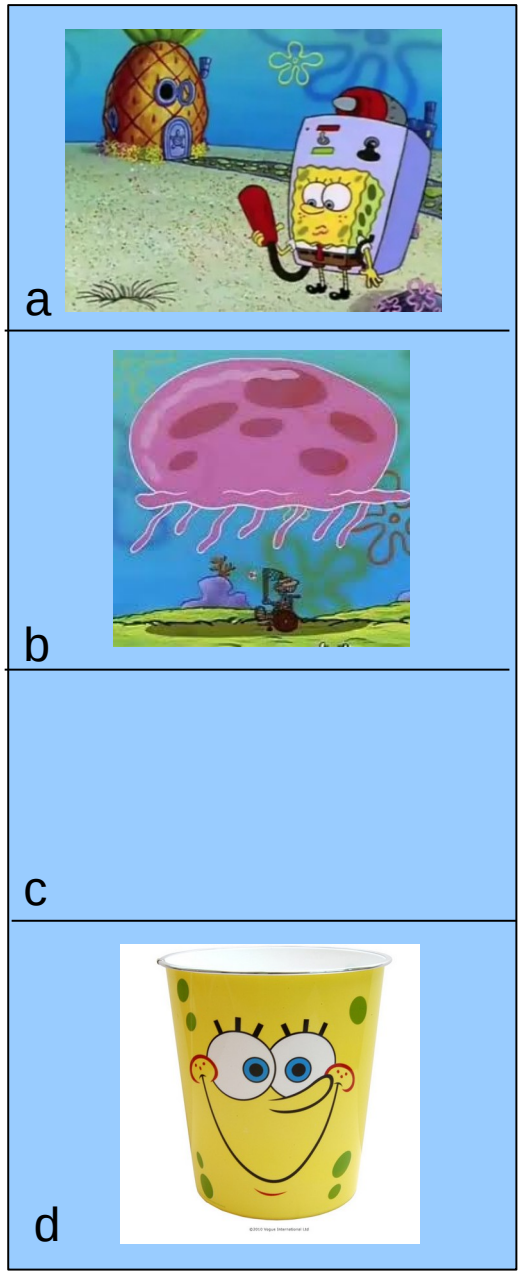
```
!bust_jellyfishes.
```

/* Plan 1 */

```
+!bust_jellyfishes :
   location(myself,X) &
   location(jellyfish,X) &
   location(bin,Y)
 <- !pick(jellyfish, X);
     move_to(Y);
     !drop(jellyfish, Y).
```

/* Plan 2 */

```
+!bust_jellyfishes :
   location(myself,X) &
   not location(jellyfish,X) &
   adjacent(X, Z)
 <- move_to(Z);
     !bust_jellyfishes.
```



Jelly-fish busters

/* Initial beliefs */

```
adjacent(a, b).
adjacent(b, c).
adjacent(c, d).
location(myself, a).
location(jellyfish, b).
location(bin, d).
```

/* Initial goal */

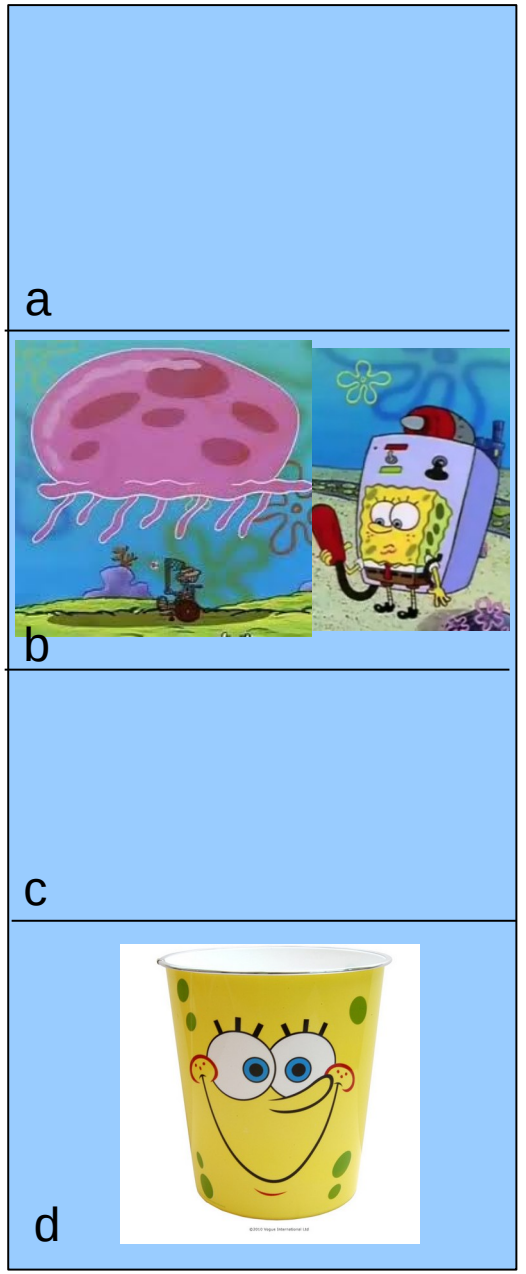
```
!bust_jellyfishes.
```

/* Plan 1 */

```
+!bust_jellyfishes :
   location(myself,X) &
   location(jellyfish,X) &
   location(bin,Y)
 <- !pick(jellyfish, X);
    move_to(Y);
    !drop(jellyfish, Y).
```

/* Plan 2 */

```
+!bust_jellyfishes :
   location(myself,X) &
   not location(jellyfish,X) &
   adjacent(X, Z)
 <- move_to(Z);
    !bust_jellyfishes.
```



Jelly-fish busters

/* Initial beliefs */

```
adjacent(a, b).
adjacent(b, c).
adjacent(c, d).
location(myself, a).
location(jellyfish, b).
location(bin, d).
```

/* Initial goal */

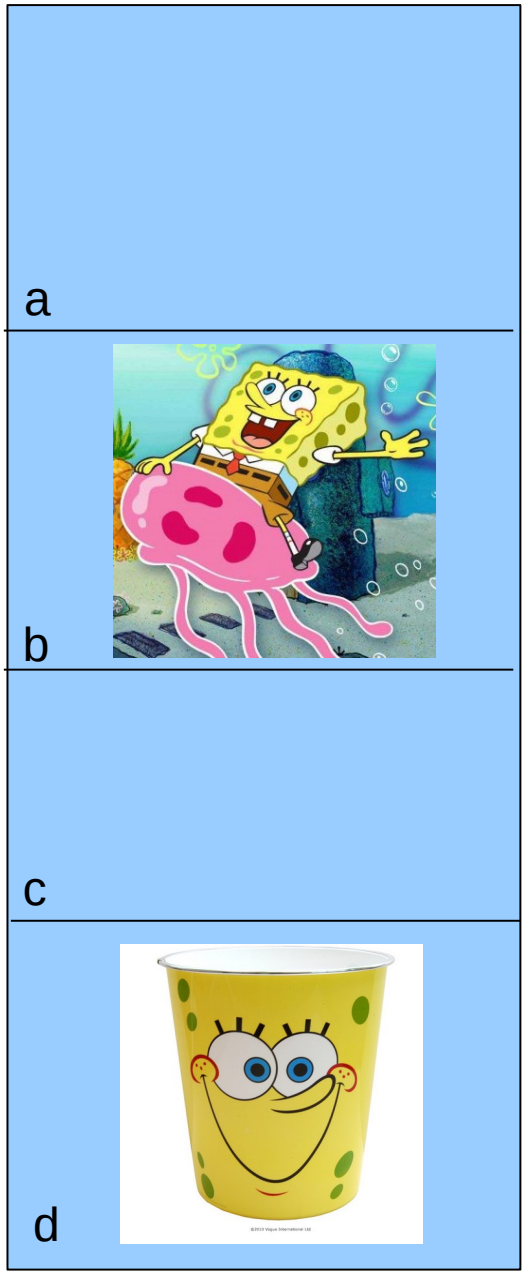
```
!bust_jellyfishes.
```

/* Plan 1 */

```
+!bust_jellyfishes :
   location(myself,X) &
   location(jellyfish,X) &
   location(bin,Y)
 <- !pick(jellyfish, X);
    move_to(Y);
    !drop(jellyfish, Y).
```

/* Plan 2 */

```
+!bust_jellyfishes :
   location(myself,X) &
   not location(jellyfish,X) &
   adjacent(X, Z)
 <- move_to(Z);
    !bust_jellyfishes.
```



Jelly-fish busters

/* Initial beliefs */

```
adjacent(a, b).
adjacent(b, c).
adjacent(c, d).
location(myself, a).
location(jellyfish, b).
location(bin, d).
```

/* Initial goal */

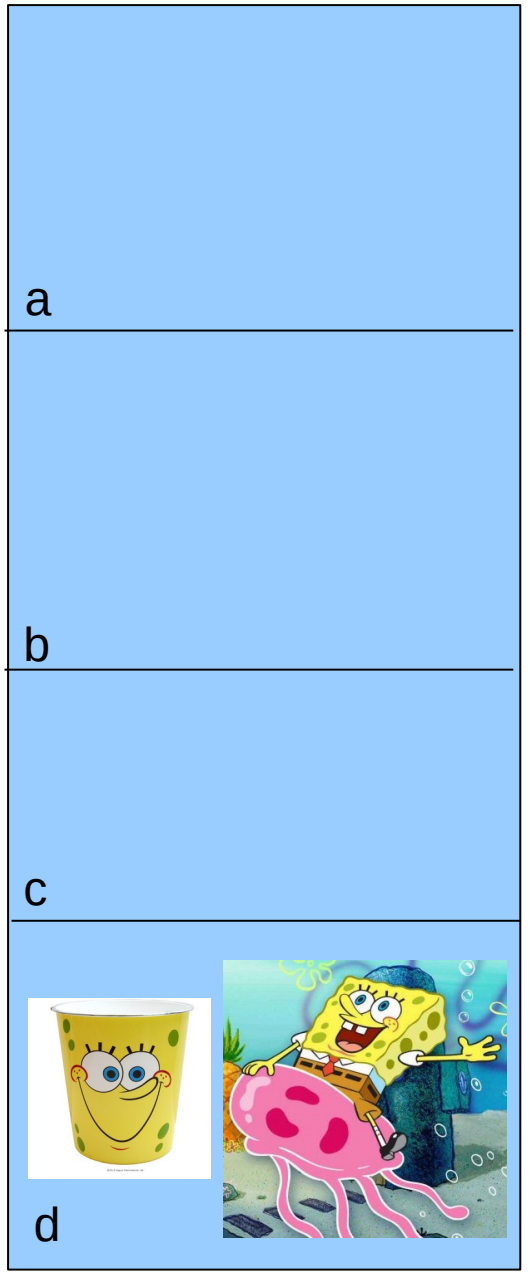
```
!bust_jellyfishes.
```

/* Plan 1 */

```
+!bust_jellyfishes :
   location(myself,X) &
   location(jellyfish,X) &
   location(bin,Y)
 <- !pick(jellyfish, X);
    move_to(Y);
    !drop(jellyfish, Y).
```

/* Plan 2 */

```
+!bust_jellyfishes :
   location(myself,X) &
   not location(jellyfish,X) &
   adjacent(X, Z)
 <- move_to(Z);
    !bust_jellyfishes.
```



Jelly-fish busters

/* Initial beliefs */

```
adjacent(a, b).
adjacent(b, c).
adjacent(c, d).
location(myself, a).
location(jellyfish, b).
location(bin, d).
```

/* Initial goal */

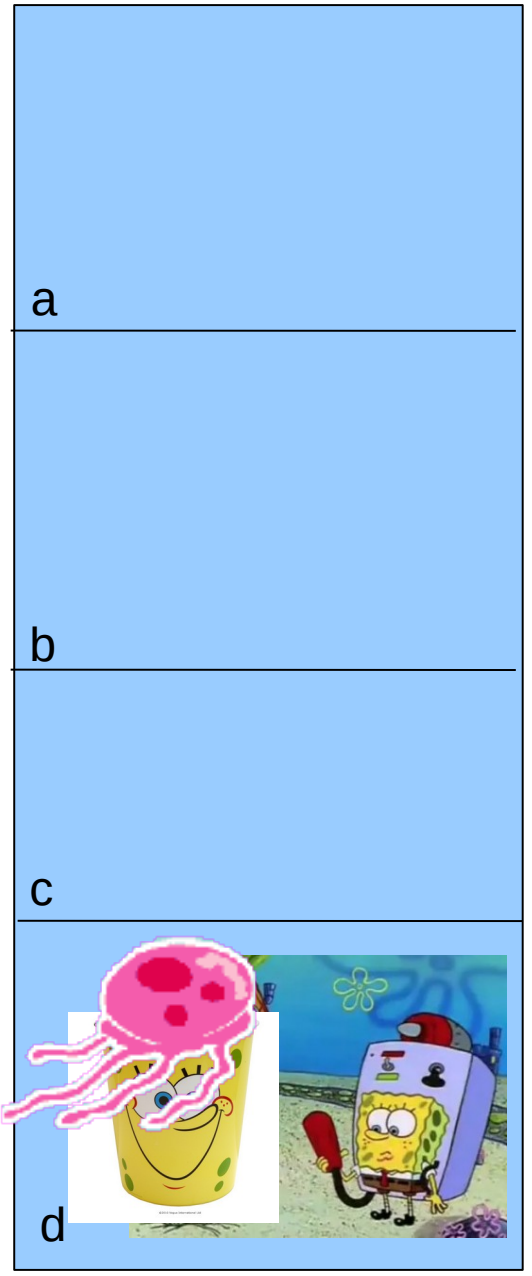
```
!bust_jellyfishes.
```

/* Plan 1 */

```
+!bust_jellyfishes :
   location(myself,X) &
   location(jellyfish,X) &
   location(bin,Y)
 <- !pick(jellyfish, X);
    move_to(Y);
    !drop(jellyfish, Y).
```

/* Plan 2 */

```
+!bust_jellyfishes :
   location(myself,X) &
   not location(jellyfish,X) &
   adjacent(X, Z)
 <- move_to(Z);
    !bust_jellyfishes.
```



Jason

```
// mars robot 1

/* Initial beliefs */

at(P) :- pos(P,X,Y) & pos(r1,X,Y).

/* Initial goal */

!check(slots).

/* Plans */

+!check(slots) : not garbage(r1)
   <- next(slot);
      !!check(slots).
+!check(slots).


+garbage(r1) : not .desire(carry_to(r2))
   <- !carry_to(r2).

+!carry_to(R)
   <- // remember where to go back
      ?pos(r1,X,Y);
      -+pos(last,X,Y);

      // carry garbage to r2
      !take(garb,R);

      // goes back and continue to check
      !at(last);
      !!check(slots).

+!take(S,L) : true
   <- !ensure_pick(S);
      !at(L);
      drop(S).
```



R.H.Bordini, J.F.Hubner, M.Wooldridge. Programming Multi-Agent Systems in AgentSpeak Using Jason. John Wiley & Sons, 2007.

Other useful material

https://jason.sourceforge.net/jBook/SlidesJason.pdf